Project Proposal: Event Management System (EMS)

Course: UE24CS341A Software Engineering, PES University, Aug to Dec 2026 **Group number:** 7 (to confirm) **Project complexity:** Medium **Technology:** React.js, Node.js (Express), PostgreSQL

Team

Member	SRN	Product ownership (feature)
Varun Rao	PES1UG24CS520	F1 Authentication and role management
Thanishq Raam Patil	PES1UG24CS502	F2 Event creation and management
Vishruta	PES1UG24CS538	F3 Event registration and ticketing
Nithin Kumar M	PES1UG25CS829	F4 Check-in, notifications and reports

Project description

Colleges, clubs and companies often manage events with spreadsheets, forms and group messages. Seats get overbooked, attendees lose their confirmations, waitlists are handled by hand and organizers have no clear count of who actually came.

The Event Management System is a web application that brings this into one place. Organizers create and publish events with a fixed capacity. Attendees browse, register and receive a ticket with a QR code. When an event is full, attendees join a waitlist and are promoted automatically, in order, when a seat is freed. At the venue, organizers check attendees in by ticket ID, and afterwards they get an attendance report and a CSV list. Email and in-app notifications keep everyone informed, and admins manage users, roles and categories.

The system is built as a layered web application (React frontend, Node.js REST API, PostgreSQL database), secured with HTTPS, hashed passwords, signed session tokens and role-based access control.

Typical users

User	What they do
Visitor	Browses and searches published events; signs up
Attendee	Registers for events, joins waitlists, views tickets and notifications, cancels registrations
Organizer	Creates, edits, publishes and cancels events; views registrations; checks attendees in; downloads reports
Admin	Manages users and roles, manages event categories, views platform totals

Functional features

F1 Authentication and role management (Varun Rao) - Sign up, log in and log out with secure sessions - Account lockout after repeated failed logins - Role-based access for Attendee, Organizer and Admin - Admin management of users and roles; profile and password change

F2 Event creation and management (Thanishq Raam Patil) - Create events as drafts with validated details, then publish them - Edit own events without reducing capacity below confirmed registrations - Cancel events, which cancels their registrations and notifies attendees - Admin management of event categories - Browse, search and filter published events with pagination

F3 Event registration and ticketing (Vishruta) - Register for an event, with no duplicate registrations - Automatic waitlist when the event is full, with first-in-first-out promotion - Cancel own registration

before the event starts - My registrations list and a ticket page with a QR code - Organizer view of the registrations for their own event

F4 Check-in, notifications and reports (Nithin Kumar M) - Check attendees in by ticket ID, rejecting invalid, used or cancelled tickets - Email notifications for confirmation, waitlist promotion, cancellation and event cancellation - Reminder email 24 hours before the event, and in-app notifications - Attendance report, CSV export of the attendee list and an admin dashboard

Out of scope: online payments, venue seat maps, native mobile apps and calendar integration.

Plan of work

The project follows Agile (Scrum) with a product backlog on GitHub, two sprints, pull-request reviews and continuous integration.

Phase	Work	Output
1. Requirements and planning	Requirements per feature, test plan, architecture and design	SRS, Test Plan, SAD (IEEE format)
2. Backlog and setup (Sprint 0)	Backlog of user stories with story points, repository, scaffold, database, CI pipeline, Docker	48 backlog issues, working build pipeline
3. Sprint 1	Core product: accounts and roles, event creation and search, registration and waitlist, emails, audit log, rate limiting	Working product, 1-minute video
4. Sprint 2	Remaining features, security and performance requirements, tests	Complete product, 2-minute video, development freeze
5. Testing and delivery	Execute all test cases, final documentation, presentation	Test results, final report, demo

Each member designs, implements and tests their own feature from end to end, and reviews another member's pull requests.